

Lab 7 – Hola!! Cassandra

Big Data

3 de agosto de 2026

Hoy, jugaremos con Apache Cassandra, un tipo de almacenamiento de datos tabular/basado en columnas. Le echaremos un vistazo al agregado y borrado de *keyspaces* (espacios de llaves), agregado y borrado de tablas, manejo de esquemas y tipos, cómo la información es insertada y consultada, etc. Usaremos la interfaz de Cassandra Query Language (CQL), que es como una versión más liviana de SQL. Aquí puedes encontrar documentación útil: <https://cassandra.apache.org/doc/4.0/cassandra/cql/>.

Copia todos los comandos a un archivo de texto para entregarlos luego a u-cursos.

Hoy día solo necesitas SSH. Para comenzar, inicia sesión en el cluster (hay que ver las instrucciones en lab 2, usuario `uhadoop`, contraseña en la pizarra).

- Puedes usar los siguientes comandos para obtener información sobre los nodos de Cassandra:

```
nodetool status
nodetool -h 10.10.10.1 info
nodetool -h 10.10.10.12 info
nodetool -h 10.10.10.13 info
nodetool -h 10.10.10.14 info
nodetool -h 10.10.10.15 info
```

- Ahora vamos a lanzar el cliente de CQL (Cassandra Query Language). Ejecuta el siguiente comando (el primer argumento es la IP del servidor, el segundo es el puerto):¹

```
cqlsh 10.10.10.1
```

(10.10.10.1 es el IP del maestro.)

- Luego vamos a crear un *keyspace* y algunas tablas.

- Primero crearemos un *keyspace* con una estrategia simple de replicación (hashing/3 copias):

```
CREATE keyspace USUARIO with replication =
{'class': 'SimpleStrategy', 'replication_factor': 3 };
```

Podemos ver la lista de *keyspaces*

```
DESCRIBE keyspaces;
```

- A continuación queremos movernos a ese *keyspace*: `USE USUARIO;`
- Luego podemos *intentar* crear una tabla en ese *keyspace* con dos columnas, una llamada `usuario` de tipo `text`, otra llamada `edad` de tipo `int`:

```
CREATE TABLE NOMBRE-ÚNICO ( usuario text, edad int);
```

Pero espera, ¡eso no funcionó! Recuerda que las tablas de Cassandra son mapas en realidad, y todo mapa necesita llaves. Necesitamos especificar una llave primaria, la cual da un identificador

¹Si hay un problema “`Connection error: ...`”, puedes intentar ejecutar:
`/data/hadoop/cassandra/bin/nodetool enablethrift`

único a cada fila:

```
CREATE TABLE NOMBRE-ÚNICO ( usuario text PRIMARY KEY, edad int);
```

Así está mejor. Nota que si no estás en el namespace (espacio de nombres) dado, aún así puedes referenciar una tabla usando `namespace.table`. Si estás dentro del namespace indicado, puedes usar simplemente el nombre de la tabla.

- Ahora podemos listar las tablas en el *keyspace* en el que estamos:

```
DESCRIBE tables;
```

Pensándolo bien, agreguemos una columna viva a la tabla:

```
ALTER TABLE NOMBRE-ÚNICO ADD viva boolean;
```

- Ok, ahora ya tienes una noción de cómo crear *keyspaces* y tablas. Después de todo el trabajo duro, ahora borrarás todo lo que acabas de hacer. ¡Por favor solo elimina tus cosas! Tendremos información más interesante si trabajamos en un keyspace/tabla común de ahora en adelante.

```
DROP TABLE NOMBRE-ÚNICO;
```

```
DROP KEYSPACE USUARIO;
```

- Movámonos a un *keyspace* común que hice yo con anterioridad, en el cual veremos cómo actualizar y consultar información.

- Primero carga el keyspace: `USE cc66i`;
- Luego, mira las tablas que hay: `DESCRIBE tables`;
- ¿Qué hay dentro de la tabla? `SELECT * FROM alumno`;

- Ahora vas a agregar tu propia información (puedes mentir). En lo que sigue, usaré mis propios datos como ejemplo. *Por favor, reemplaza mis datos con los tuyos*. Por favor, tampoco sobrescribas mi fila. Si lo haces me pondré triste.

- Primero vamos a insertar un nombre con iniciales:

```
INSERT INTO alumno( usuario, nombre ) VALUES ( 'amoya', 'A. Moya');
```

Consulta la tabla de nuevo para asegurarte que quedó ahí: `SELECT * FROM alumno`;

- ¿Qué pasa si intentamos insertar un tipo incorrecto?:

```
INSERT INTO alumno( usuario, nombre ) VALUES ( 'amoya', true);
```

(¡No funciona! El tipo de `nombre` es text, no boolean.)

- Ahora tratemos de dar nuestro nombre completo en vez:

```
INSERT INTO alumno( usuario, nombre ) VALUES ( 'amoya', 'Alberto Moya');
```

Si consultas la tabla de nuevo verás que el valor antiguo está sobrescrito. A diferencia de BigTable/Dynamo, etc., los valores no son versionados en Cassandra.

- Intentemos agregar más información, como nuestra edad:

```
INSERT INTO alumno( edad ) VALUES ( 37 );
```

¿No funciona? ¿Por qué no? (¿De quién es la edad que deberíamos actualizar? ¡Necesitamos especificar la llave!)

```
INSERT INTO alumno( usuario, edad ) VALUES ( 'amoya', 37 );
```

Intenta consultar la tabla de nuevo. Vemos que la edad fue agregada junto al nombre.

- Completa el resto de los detalles en la tabla por ti mismo siguiendo la misma idea anterior.

- Ok, ahora hagamos algunas consultas sobre la tabla; esperemos que se esté llenando.

- Obtengamos el número de personas en la tabla:

```
SELECT COUNT(*) AS count FROM alumno;
```

- El nombre de un usuario específico:

```
SELECT nombre FROM alumno WHERE usuario = 'amoya';
```

- Personas que tienen una edad específica:

```
SELECT nombre FROM alumno WHERE edad = 37;
```

 Oops, no funcionó.
- En realidad estamos muy limitados en cuanto a lo que podemos hacer por defecto en Cassandra. Como cualquier mapa, solo podemos hacer búsquedas por llave: en este caso, la llave primaria es el nombre de usuario. Para poder hacer más, necesitamos crear índices.
 - Como primer paso, deberías hacer una copia personal de la tabla, guardándola en un archivo y volviendo a cargarla (aquí está todo en un solo comando):

```
COPY alumno ( usuario, color_fav, comentario, despierto, edad, nombre, pelicula_fav ) TO '/camino/a/tu/carpeta/[USUARIO].csv';
```
 - Si quieres revisar el archivo:

```
exit
```

```
more [USUARIO].csv
```

 .. y para regresar:

```
cqlsh 10.10.10.1 9160
```

```
USE cc66i;
```
 - Ahora (1) **crea una nueva tabla** con tu nombre de usuario que corresponda a los datos y (2) cárgalos desde el archivo:

```
COPY [USUARIO] ( usuario, color_fav, comentario, despierto, edad, nombre, pelicula_fav ) FROM '/camino/a/tu/carpeta/[USUARIO].csv';
```

 Ésta es tu propia copia de la tabla para que puedas jugar.
 - Digamos que estamos construyendo una aplicación que necesita realizar búsquedas de usuarios con una edad específica. Para ello, en Cassandra necesitamos crear un índice.

```
CREATE INDEX edadIndex[USUARIO] ON [USUARIO] (edad);
```
 - Ahora podemos tratar de buscar usuarios por edad:

```
SELECT nombre FROM [USUARIO] WHERE edad = 37;
```

 Ok, genial, ¡ahora funciona!
 - Ahora descubre si puedes hacer funcionar la siguientes consultas (échale un vistazo al link a la documentación dado al comienzo del enunciado):

```
SELECT nombre FROM [USUARIO] WHERE edad = 37 AND despierto = true;
```

```
SELECT nombre FROM [USUARIO] WHERE edad > 25;
```
- Para el objetivo del lab, está bien si la última consulta arroja un *warning*. Pero es posible hacerla sin un *warning* en Cassandra usando tablas con llave primarias particulares (una llave compuesta). El tipo de llave más complejo en Cassandra tiene la forma PRIMARY KEY ((pk1, ..., pkn), ck1, ..., ckn). Como en una base de datos relacional, la llave entera debería tener una sola tupla para un valor de pk1, ..., pkn, ck1, ..., ckn. Adicionalmente, el valor de (pk1, ..., pkn) (llave de partición) será usado para particionar la relación: para elegir cuál máquina guardará la tupla (ej, usando un *hash* de (pk1, ..., pkn)). Después, en cada máquina, ck1, ..., ckn (llave de *clustering*) será usado para ordenar los datos locales de cada máquina, ordenados por ck1, ck2, etc. Una llave de partición es obligatoria, pero como vimos antes, la llave de *clustering* es opcional. En una definición PRIMARY KEY (pk, ck1, ..., ckn), por defecto, la primera columna será la llave de partición y las otras columnas será la llave de clustering.
- Para una llave PRIMARY KEY ((pk1, ..., pkn), ck1, ..., ckn), una consulta tiene que dar todos los valores en pk1, ..., pkn. Opcionalmente puede dar algunos valores en ck1, ..., ckn pero en orden de 1 ... n sin brechas. Por ejemplo, se puede consultar por valores de (pk1, ..., pkn) o (pk1, ..., pkn, ck1) o (pk1, ..., pkn, ck1, ck2), etc., pero no con pk1 o (pk1, ..., pkn, ck2), etc.

- Crea una tabla con una llave primaria que puede soportar las siguientes consultas sin un *warning*. De nuevo, se pueden cambiar los valores exactos.
 - `SELECT * FROM

TABLE2NAME

 WHERE color_fav='black';`
 - `SELECT * FROM

TABLE2NAME

 WHERE color_fav='black' AND edad>20;`
 - `SELECT * FROM

TABLE2NAME

 WHERE color_fav='black' AND usuario='amoya' AND edad=37;`
- Para terminar la sesión: `exit`
- ENTREGA a U-cursos la secuencia de comandos empezando con `CREATE KEYSPACE ....`